

Phased Migration Plan for Shared SDK Automations

The main goal is to centralize our shared logic step-by-step. At the same time, individual SDKs will keep control over their own workflow permissions, checkouts, and security settings until each new automation is fully battle-tested.

Supporting plan: architectural migration strategy, phased execution timeplan, and community issue rollout.

Status: strategy draft for maintainer review

Related work:

- LFDT mentorship project:
<https://github.com/LF-Decentralized-Trust-Mentorships/mentorship-program/issues/73>
- C++ investigation issue: <https://github.com/hiero-ledger/hiero-sdk-cpp/issues/1627>
- Central automation candidate repo: <https://github.com/darshit2308/sdk-automations>
- Python canary fork: <https://github.com/darshit2308/hiero-sdk-python>
- Central repo CI proof:
<https://github.com/darshit2308/sdk-automations/actions/runs/25763437710>
- Python fork canary proof:
<https://github.com/darshit2308/hiero-sdk-python/actions/runs/25763950499/job/75671903429>

Planning Readout For Mentor And Maintainer Review

Direct Answers To The Four Core Planning Questions

What is the strategy? The strategy is to migrate behavior family by behavior family through a shared core, not to rewrite every SDK workflow at once. Each automation is classified first, then only the duplicated decision logic moves into the central repo, while triggers, permissions, checkout, harden-runner, and repo policy remain local until parity tests, dry-run evidence, and pilot observation prove the shared path is trustworthy.

What can we reuse? We are looking for tasks that are identical across SDKs and time-consuming to manage separately, things like the logic for syncing reviews, common GitHub API helpers, configuration checks, and standard labeling rules. We use a simple "three-bucket" rule to decide: only the code that truly belongs in a shared core moves to the central repo, while security-sensitive and repository-specific settings stay exactly where they are.

What gets retired or torn up? Local code is retired only when the shared replacement is already fixture-backed, dry-run validated, pilot-proven, and maintainer-approved. What stays local permanently are the security and trust boundaries: workflow YAML, permissions, checkout, concurrency, artifacts, secrets, and protected repo config. What gets replaced gradually are duplicate decision trees and helper code. What should not be torn up yet are C++ contributor-facing assignment flows, workflow_run edge cases, and unresolved repo-specific policy logic.

The solution is built for the future because the shared core doesn't care how it's triggered. While it powers GitHub Actions today, it's designed to work with Probot or a dedicated GitHub App later without needing a rewrite. This keeps our initial setup simple while laying the groundwork for a more robust architecture where multiple SDKs reuse the same tested logic.

LFDt 2026 Mentorship Framework

This roadmap serves as a six-month LFDt mentorship plan (June 15 – November 30, 2026). With pre-mentorship proofs already established, including the central action model and Python fork canary, the mentorship begins from a validated starting point, correctly scoping C++ as design-first work.

Three-Bucket Classification: The Main Decision Rule

The success of this migration depends on a simple rule: every part of an automation must be placed into one of three buckets before we start refactoring.

1. **Shared core candidate:** duplicated decision logic that should move into sdk-automations and be tested once for reuse across SDKs.
2. **Repo-local boundary:** workflow triggers, permissions, checkout, harden-runner, concurrency, artifacts, and other security or orchestration wiring that should remain in each SDK repository.
3. **Repo-specific policy:** labels, teams, thresholds, docs links, and behavioral policy values that should stay local, but be expressed through a protected config/schema contract rather than scattered hardcoded constants.

This three-bucket model is the core strategic filter for the whole document. It answers what is reusable, what stays local, and what should never be centralized just for aesthetics.

This isn't meant to be a long-winded documentation project. We've designed it to be practical: the first half focuses on shipping a working shared action for maintainers to review, while the second half focuses on hardening the more complex features and planning for future growth. In concrete terms, a strong midterm should already show a stabilized central repo, parity-backed review-sync core, and a small upstream Python dry-run PR. A strong finish should show not only assignment readiness and rollout discipline, but also an app-ready core with a minimal Probot or GitHub App adapter spike if the earlier gates land cleanly.

Measurable Mentorship Success Criteria

Committed scope vs. stretch scope

Committed scope for the six-month program: stabilize the central repo under a production-quality governance model, prove Python review-sync end-to-end, build parity-backed shared logic, map assignment behavior rigorously, and leave behind a maintainer-reviewed rollout package for C++ and wider adoption.

Stretch scope if the earlier phases finish on time: start a minimal Probot or GitHub App readiness spike in the final month. That spike should stay thin and honest: adapter contract,

event-routing surface, read-only or low-risk proof, and deployment/runbook outline. It should not become a second implementation or a rushed production-hosting commitment.

This distinction matters because the plan should look ambitious without becoming brittle. Full hosted app ownership is not the first milestone; proving the core in a form that can feed both Actions and a later App is.

Metric	Target by the end of the program	Evidence expected
Maintenance friction	At least one shared automation rule can be updated centrally instead of through repeated per-repo patches	Central repo change plus repo-level adoption evidence
Pilot safety	No contributor-facing disruption during the Python pilot and no silent regression in review-sync behavior	Pilot logs, rollback drill, maintainer sign-off
Testing discipline	90%+ unit/integration coverage for state-mutating shared logic and adapters touching live behavior	Coverage reports and CI gates
Community leverage	A structured issue backlog exists for contributors with clear safe-to-open phases	Issue map, labels, and phase-linked backlog
C++ decision quality	C++ ends the program with a concrete recommendation package rather than speculative abstraction	Before/after examples, behavior matrix, and maintainer-reviewed recommendation

Mentorship success is measured by the creation of safe, maintainable, and extensible workflows that actual maintainers can easily inherit and manage.

This document is intended to answer planning questions, not only technical feasibility questions.

A mentor or maintainer should be able to read this section first and understand how the migration would actually move, who would own each stage, and when the work becomes safe to open to the community.

- What is the architecture boundary between SDK-local workflow wiring and the central shared-logic repository?

- Why is the rollout ordered Python first, C++ later, and Probot/GitHub App after the shared action model is proven?
- What does each phase produce, and what evidence or decision gate closes that phase?
- Which tasks should stay maintainer-owned, and which tasks can be opened to community contributors without creating security or review risk?
- What is the first 4-6 week execution sequence if maintainers decide to move forward?
- Which decisions are still explicit dependencies rather than assumptions baked into the plan?

Strategic Phasing vs. Big-Bang Migration

A broad migration is risky given the varying maturity levels and workflow shapes across the ecosystem. Phased execution allows maintainers to validate integration points, such as local configuration loading and dry-run safety, before production deployment.

A phased migration is safer because it lets maintainers answer one question at a time:

- Can an SDK repo call a central action without losing local workflow control?
- Can the action load repo-local config correctly?
- Can it run safely in dry-run mode?
- Can it mutate only a fork or controlled pilot before production?
- Can the central core be tested independently of GitHub Actions globals?
- Can parity tests prove expected behavior before upstream writes are enabled?
- Can rollback stay simple?
- Can C++ be evaluated without disrupting its existing contributor-facing bot behavior?

The first proof should be small and real, not large and speculative. Python `review-sync` is the best first candidate because it is bounded, already related to review queue automation, and lower risk than assignment logic.

Current Evidence

Central repo proof

The candidate central repository is:

<https://github.com/darshit2308/sdk-automations>

Current structure includes:

- `packages/core`: shared automation logic.
- `packages/github-action-adapter`: GitHub Actions input/client boundary.
- `actions/review-sync`: callable review-sync JavaScript action.
- `actions/run-automation`: generic dispatcher action.
- `schemas/hiero-automation.schema.json`: repo config schema.
- `examples/`: caller examples for SDK repositories.

- `docs/`: architecture, caller interface, StepSecurity notes, and migration docs.

The central repo has CI that runs tests, builds action bundles, and verifies that committed `actions/*/dist/index.js` bundles are current(updated):

<https://github.com/darshit2308/sdk-automations/actions/runs/25763437710>

The large `dist/index.js` files are generated GitHub Action bundles, not hand-written source. They are committed so SDK caller repositories can use the action without installing this repo's workspace dependencies.

This repo is still candidate proof infrastructure, not the final upstream dependency target. Before any official SDK consumes a central action in production, the central repo needs official ownership, protected-branch governance, release ownership, and incident/rollback ownership.

Python canary proof

The Python fork canary is:

<https://github.com/darshit2308/hiero-sdk-python>

The canary adds only:

- `.github/hiero-automation.yml`
- `.github/workflows/central-review-sync-canary.yml`

The canary proves that an SDK repository can keep workflow ownership local while calling reusable logic from the central repo.

The workflow keeps these pieces in the Python SDK fork:

- `workflow_dispatch` trigger
- `permissions`
- `step-security/harden-runner`
- checkout
- config file path
- token choice
- dry-run/write mode choice

A successful fork run proved the central action can execute end-to-end and apply the expected review labels in the fork:

<https://github.com/darshit2308/hiero-sdk-python/actions/runs/25763950499/job/75671903429>

C++ investigation status

The C++ issue is intentionally scoped as an investigation, not a production workflow refactor:

<https://github.com/hiero-ledger/hiero-sdk-cpp/issues/1627>

The issue asks for:

- target automation prototype identification
- caller interface definition
- StepSecurity impact analysis
- simple C++ workflow before/after
- awkward C++ workflow before/after
- final recommendation

- no production workflow behavior changes as part of the investigation

This plan aligns with that scope by recommending central action/core extraction first and C++ workflow changes later.

Repository Landscape

The Hiero SDK ecosystem does not have one uniform automation shape. Some SDKs mostly have build/release workflows. Python and C++ have much larger local automation surfaces. The following matrix was sampled from public GitHub repository trees on 2026-05-13.

Repository	Primary language	Workflow files	Local <code>.github/scripts</code> files	Local <code>.github/actions</code> files	Migration implication
<code>hiero-sdk-python</code>	Python	47	66	0	Best first pilot. Large automation surface, but review-sync is bounded.
<code>hiero-sdk-cpp</code>	C++	12	46	0	High-value but sensitive. Needs investigation and behavior mapping before migration.
<code>hiero-sdk-js</code>	JavaScript	14	0	0	Later candidate after shared model is stable.
<code>hiero-sdk-java</code>	Java	7	0	0	Later candidate, likely workflow-level adoption only at first.
<code>hiero-sdk-tck</code>	TypeScript	8	0	0	Later candidate, compatibility workflow focus.

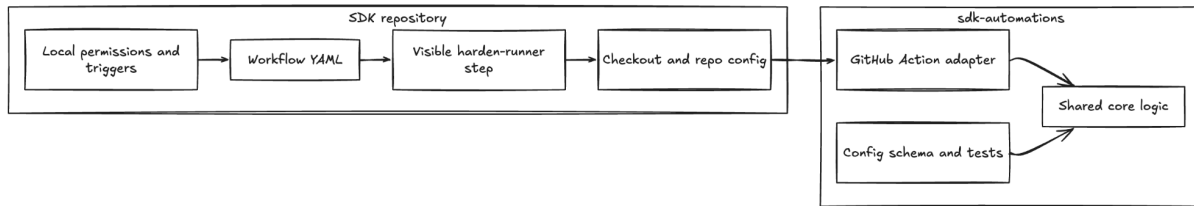
Repository	Primary language	Workflow files	Local <code>.github/scripts</code> files	Local <code>.github/actions</code> files	Migration implication
<code>hierosdk-rust</code>	Rust	3	0	0	Later candidate, lower automation surface.
<code>hierosdk-go</code>	Go	1	0	0	Later candidate, likely only if shared policy becomes relevant.
<code>hierosdk-swift</code>	Swift	1	0	0	Later candidate, low automation surface.
<code>hierodid-sdk-python</code>	Python	4	0	2	Later candidate, already has a local actions pattern.
<code>hierodid-sdk-js</code>	TypeScript	4	0	2	Later candidate, already has local actions pattern.

This matrix supports the rollout order:

1. **Python first** because it has the largest automation surface and a bounded review-sync candidate.
2. **C++ second** because it has **high-value contributor bot logic** but **higher risk**.
3. **Other SDKs later** because they do not yet show the same local bot-script duplication pattern.

Architecture Strategy

The target architecture is actions-first and Probot-later.



What remains local in SDK repositories

SDK repositories keep:

- workflow triggers
- `permissions`
- concurrency groups
- `if:` guards
- runner selection
- checkout strategy
- artifact upload/download wiring
- `step-security/harden-runner`
- repository secrets and tokens
- repository-owned config files
- multi-job orchestration

These are local because they define the security boundary and event context for each repository.

What moves into `sdk-automations`

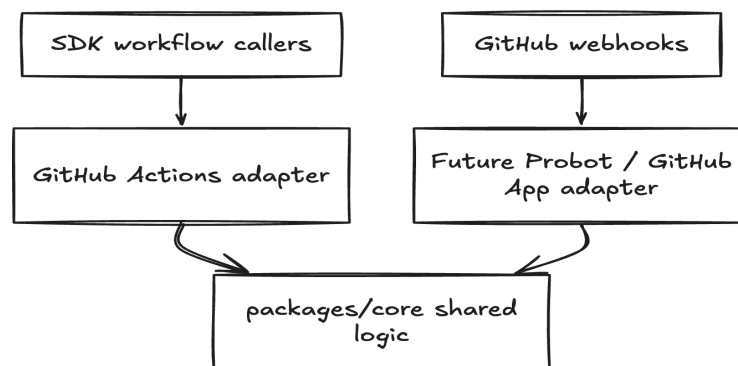
The central repo owns:

- shared automation core logic
- config schema and validation
- GitHub API adapter boundaries
- reusable comment/label/review decision logic
- tests and fixtures
- action bundles
- examples and rollout docs
- future Probot/GitHub App adapter code

The central repo should not own every caller workflow. It should own behavior that is actually duplicated and expensive to maintain across repositories.

Why Probot follows the shared core

- While a GitHub App or Probot service is great for the long term, they bring extra work like hosting and incident response. Starting with JavaScript Actions is much safer because SDKs can adopt them one small step at a time without the overhead.
- That sequencing should not be read as “ignore Probot for six months.” It means “do not make hosted app operations the first dependency.” If Phases 1-3 are green on schedule, the final stretch of the mentorship should still aim to produce at least a minimal app-adapter spike so the future path is concrete, not hypothetical.
- The right end-state for the mentorship is therefore: Actions-first for the first production proof, with an app-ready shared core and a thin adapter contract that makes a later Probot deployment straightforward rather than speculative.
- The long-term design still supports Probot:



The key is that Probot should reuse the same core modules. It should not become a second copy of the logic.

Code Migration Mechanics: What Moves, What Stays, What Gets Retired

- This section explains exactly how we'll move the code. We aren't going to sweep everything out of the SDK repos at once.
- Instead, we'll use a "strangler" approach: we'll inventory what's there, move only the shared decision-making logic into the core, and only retire the old local code once we've proven the new system works perfectly.

How the code actually moves

1. Inventory the current automation by behavior family rather than by filename. The important families are review-sync, assignment, repo policy/config, GitHub API helpers, workflow orchestration, test fixtures, and repo-only special cases.
2. Classify each behavior into one of three buckets: shared core candidate, repo-local boundary, or repo-specific policy. Shared core candidates are the only things that move to

sdk-automations. Repo-local boundary code stays in the SDK repo. Repo-specific policy values move out of code and into protected repo config, not into a globally hardcoded module.

3. Recreate current behavior in tests before replacing it. For each candidate behavior, import or rebuild golden fixtures from the current Python or C++ implementation so the shared core has an automated parity target instead of only a human review target.

4. Extract the shared decision logic into packages/core behind a small adapter boundary. The adapter should take trusted inputs such as repo config, event payload, and GitHub API client, and return explicit actions, comments, or label decisions.

5. Add a thin repo-local caller workflow and config file. The caller keeps permissions, checkout, harden-runner, concurrency, if guards, and any artifact orchestration, while invoking the central action only for the shared logic.

6. Pilot one behavior upstream at a time. Python review-sync is the right first pilot because it is lower-risk than contributor-facing assignment behavior and it proves the caller contract without forcing a C++ restructuring.

7. Retire duplicated local implementations only after three gates are passed: parity tests are green, upstream dry-run evidence matches expectations, and a write-mode pilot completes its observation window with a documented rollback path.

Reusability matrix

The table below is the migration filter. It makes the future code movement explicit, and it prevents the project from centralizing code simply because it exists in more than one repository.

What gets torn up, replaced, or retained

- **Retain:** workflow triggers, permissions, checkout strategy, harden-runner, concurrency rules, artifact wiring, secrets, repo-owned config, and multi-job structure. These are not temporary scaffolding; they are the permanent local boundary even in a successful migration.
- **Replace gradually:** duplicate review-sync decision code, shared API helper code, ad-hoc config validation, and other repeated label/comment decision trees. These are the best early candidates to centralize because they create real maintenance drag across repositories.
- **Retire only after proof:** old local review-sync implementations in the pilot repo, duplicate helper functions that become thin wrappers over the shared action path, and fixture sets that the central repo can own once parity is proven. Retirement should happen only after parity CI, successful pilot observation, and maintainer sign-off.
- **Do not tear up yet:** C++ contributor-facing assignment workflows, complex workflow_run artifact flows, multi-job edge-case orchestrations, and any repo-specific logic with unresolved policy differences. Those stay local until mapping work proves exactly what is shared and what is policy.
- **The practical rule is simple:** if removing a local file would also remove a local trust boundary, event boundary, or unresolved repo policy, it is too early to remove that file.

Why this remains future-looking rather than overfit

- **Future-looking answer in one sentence:** the architecture builds one tested shared core that can power GitHub Actions first and Probot or a GitHub App later, so the orchestration layer can change without rewriting the automation logic.
- Future-looking here means stable contracts rather than early over-abstraction. The plan stays forward-compatible because shared core modules are orchestration-agnostic: GitHub Actions now and Probot or a GitHub App later can both reuse the same decision logic instead of creating a second implementation.
- It is also future-looking because repo policy is externalized into a versioned schema-backed config contract. That means a new SDK adopts the shared model by mapping its policy into protected config and a small local caller workflow, not by copying another repository's scripts line-for-line.
- Finally, the plan is future-looking because it opens the right community surface: fixtures, schema docs, behavior matrices, rollout checklists, repo mappings, and parity tests can be parallelized safely before privileged workflow rewiring is opened more broadly.
- In other words, the design prepares for more SDKs, more automation families, and a later Probot layer without forcing the project to over-centralize too early.

Code family	Reusable now?	Central destination	What stays local	Retirement / timing rule
Review-sync decision rules, label routing, reviewer queue logic	Yes	packages/core + review-sync action adapter	Trigger, permissions, checkout, harden-runner, repo config	Retire local pilot implementation only after parity CI, accepted dry-run evidence, and stable write pilot
Shared GitHub API helper logic and normalization	Yes	packages/core adapter/helper layer	Token scope, permission declaration, caller workflow context	Remove duplicated helpers as each caller is switched to the shared action path
Config schema and validation	Yes, but values stay local	Central schema + validator	Actual labels, teams, limits, docs links, and repo policy files	Replace ad-hoc validation once repos adopt the protected schema contract

Workflow YAML, triggers, concurrency, checkout, harden-runner	No	Does not move	All of it stays in each SDK repo	Never centralize just to reduce YAML; this is the permanent local security boundary
workflow_run artifact handling and multi-job orchestration	Usually no, unless clearly duplicated later	Possible future helper docs only	Artifact wiring, trust checks, multi-job structure	Revisit per workflow after maintainers explicitly approve a narrower shared boundary
Assignment state machine, comments, limits, and transitions	Later, after mapping	Future packages/core assignment module	Current C++/Python assignment implementations and repo policy differences	Do not retire local code until Phase 4 mapping, fixture parity, and maintainer review are complete
Repo-specific policy constants, labels, teams, thresholds	Shape is reusable; values are not	Central schema contract only	Protected .github/hiero-automation.* values in each repo	Move values from code to protected config, not into a shared hardcoded file
Repo-only special cases (release checks, coderabbit flows, one-off bots)	Case-by-case	Only if real duplication is later proven	Local script/workflow ownership	Keep local until there is a concrete shared behavior and a real maintenance benefit
Probot / GitHub App orchestration	Later adapter, not first migration	Future adapter over the same core	Hosting, app credentials, incident handling	Only add after the shared action model is stable and officially owned

Phased Execution Plan and Timeplan

How To Read This Roadmap: the integrated table below is the calendarized version of the engineering phases. Phases describe what truth must be established; the timebox column shows when that truth should be reviewed during the June-November LFDT program.

Roadmap Structure and Mapping

Planning layer	What it means	How to read it in this document
Phase 0	Pre-mentorship proof work	Already completed before June 15; proves the shared-action direction is worth pursuing
Phases 1-5	Core mentorship engineering gates	These are the main in-program execution phases that map to Milestones 1-5 from June through mid-November
Midterm checkpoint	Evaluation point after the early in-program phases	Used to review whether the shared-action model, testing evidence, and pilot readiness are strong enough to continue
Phases 6-8	Stretch or post-mentorship expansion path	Prepared during the mentorship and pulled partly into late-program weeks if earlier gates are green; otherwise continued after Nov 30
Milestones	Calendarized wrappers around the phases	These give dates, expected deliverables, and review gates for the corresponding phases

Phase 0: happens before the official program starts.

Phases 1-5: are the committed June-November mentorship scope.

Phases 6-8: are the stretch or continuation path: they should be prepared during the mentorship, and the plan should try to pull the smallest viable piece of them into the late program window if earlier gates are green, but they must not weaken the quality bar of the core phases.

Acceleration path if earlier phases land early

If the governance work, Python dry-run, and controlled pilot all land cleanly by or before the midterm checkpoint, the program should not simply slow down. The late-program window should then be used to start one carefully bounded stretch path: either a minimal Probot or GitHub App adapter spike, or a tightly scoped next-repo canary, depending on maintainer priority.

That keeps the plan from looking timid while still respecting the dependency order. The rule is: accelerate only by adding bounded next-step proofs, never by cutting testing, parity, or security gates from the committed phases.

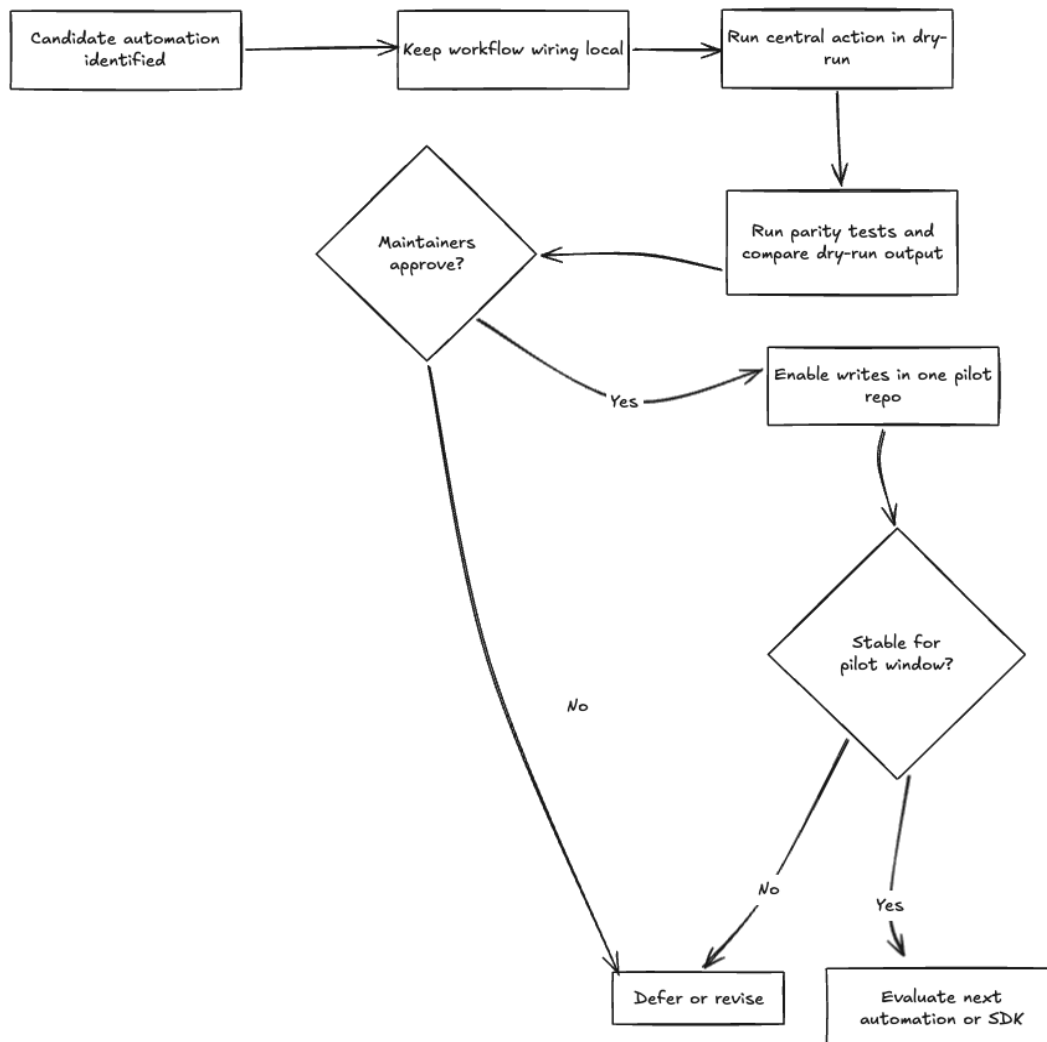
Integrated Phase-Milestone Roadmap

The table below is the single source of truth for the roadmap. It combines phase number, milestone timing, owners, deliverables, and review gates so readers can understand the sequence in one pass.

Timebox	Phase / milestone	Primary focus	Concrete deliverables	Review gate / owner
Now - Jun 14	Phase 0 / Pre-mentorship proof	Validate the shared-action direction before the program starts.	Central repo prototype, Python fork canary, updated design framing, C++ issue alignment.	Mentor/maintainer review confirms the direction is worth pursuing; no upstream SDK depends on the candidate repo yet.
Jun 15 - Jul 15	Phase 1 / Milestone 1	Stabilize the central repo foundation and governance model.	Official ownership decision, branch protection/COD EOWNERS expectations, release/pinning policy, config-protection rules, schema/versioning contract, parity harness scaffold.	Central repo maintainers + author. Gate: governance path is explicit and config/schema tests pass.
Jul 16 - Aug 23	Phase 2 / Milestone 2	Prove Python review-sync through an	Stabilized review-sync action, imported	Python maintainers + author. Gate:

		upstream dry-run pilot.	parity fixtures, small Python upstream dry-run PR, adoption checklist.	90%+ review-sync core/adaptor coverage, parity fixtures pass, dry-run output is reviewable.
Aug 24 - Aug 31	Midterm checkpoint	Review whether the shared-action model is ready for controlled writes.	Live demo, dry-run results, parity evidence, rollback path, first community-safe issues.	Mentors and maintainers decide whether to continue to write mode or narrow scope.
Sep 1 - Sep 30	Phase 3 / Milestone 3	Run a controlled Python review-sync write pilot.	Named pilot owner, concurrency rules, rollback SLA, audit/logging view, controlled write-mode pilot.	Python maintainers + pilot owner. Gate: observation window completes without unrelated label churn or maintainer disruption.
Oct 1 - Oct 31	Phase 4 / Milestone 4	Map assignment behavior before sharing assignment code.	Python/C++ behavior matrix, comment/transitions on inventory, config mapping, simple and awkward C++ before/after examples.	Maintainers agree what is shared, what remains repo-specific, and what must still wait.
Nov 1 - Nov 14	Phase 5 / Milestone 5	Prepare shared assignment-core and rollout package.	Assignment fixture harness, validation helpers, community issue backlog, wider-adoption checklist, C++	Author + maintainers + contributors. Gate: shared behavior is fixture-backed before any production C++

			recommendatio n package.	change is considered.
Nov 15 - Nov 30	Final handoff + stretch path	Package the work and start a bounded stretch proof only if earlier gates are green.	Maintainer docs, contributor guidance, recorded demo, backlog, adoption checklist, optional Probot/GitHub App adapter spike or event contract.	Final review judges this as a maintainable continuation path, not a one-off experiment.
After Nov 30, or earlier only if everything goes as planned.	Phases 6-8 / continuation	C++ canary, wider SDK rollout, and Probot/GitHub App adapter.	Prepared next-step issues, rollout criteria, app-readiness news, and per-SDK adoption checklist.	Org maintainers + per-SDK maintainers. Gate: no expansion weakens testing, parity, security, or local workflow ownership.



Phase 0: Pre-Mentorship Proof

Goal

Show that the strategy is more than a proposal. Prove that a central action can be called from an SDK repository while local workflow boundaries remain visible.

What changes

Already completed in the candidate repos:

- `sdk-automations` has shared core/action structure.
- `sdk-automations` has CI and bundled action artifacts.
- Python fork has `.github/hiero-automation.yml`.
- Python fork has `central-review-sync-canary.yml`.

- Python fork can call the central action.

What stays local

In the Python fork:

- trigger
- permissions
- harden-runner
- checkout
- config file
- token choice
- write/dry-run mode

Acceptance gate

Phase 0 is accepted when:

- the central repo CI passes
- the Python fork canary workflow starts and completes
- harden-runner remains visible in the caller workflow
- the workflow calls the central action rather than a local script
- the diff is limited to config and canary workflow files

Rollback plan

Delete the canary workflow and config from the fork. **No upstream production repo is affected.**

Maintainer review required

Informal review only. This is a fork proof, not an upstream production proposal.

Evidence produced

- central repo CI run
- Python fork canary run
- Python fork workflow file
- Python fork config file

Phase 1: Central Repo Stabilization

Goal

Make `sdk-automations` safe enough for an upstream dry-run PR.

What changes

Add or verify:

- official repository ownership or an explicitly approved long-term home under project governance
- protected default branch
- CODEOWNERS covering workflow-adjacent docs, schemas, and config guidance
- named release owner(s)
- named incident owner
- named rollback owner
- release tags for human-readable release tracking
- release notes explaining supported automation and known limits
- permissions documentation per action
- rollback documentation
- schema validation documentation
- schema and action input versioning policy
- behavior-parity fixtures and golden tests for Python review-sync (ensure new system == old system)
- CI gate that verifies generated action bundles are current (dist/index.js)
- short note explaining that `dist/index.js` is generated

What stays local

SDK repositories still own all workflows. No upstream SDK workflow changes are required in this phase.

Acceptance gate

Phase 1 is accepted when:

- central CI passes
- the central repo has official ownership and governance approval
- the default branch is protected and CODEOWNERS is in force for critical files
- action bundles are committed and current
- docs explain caller permissions
- docs explain full-SHA production pinning
- docs explain rollback
- docs explain schema and input compatibility policy
- Python review-sync parity fixtures pass in CI
- at least one official release path exists for deriving approved SHAs

Rollback plan

Do not adopt the action upstream until this phase passes. If a bad central release is created, revoke or deprecate that release, publish a new approved SHA, and keep upstream SDKs pinned to the **last known-good SHA**.

Maintainer review required

Review from the automation mentor or maintainers who will evaluate the first upstream pilot.

Evidence produced

- governance owner list
- release tag and approved production SHA
- CI run
- release notes
- docs updates
- parity test results

Phase 2: Python Upstream Dry-Run Pilot

Goal

Open a small, low-risk PR to `hieroglyphics-python` that proves the central action can run from upstream without replacing existing automation.

What changes

Add only:

- `.github/hieroglyphics-automation.yml`
- `.github/workflows/central-review-sync-canary.yml`

The workflow should be manual first:
on:

`workflow_dispatch:`

The action should run with dry-run enabled. (dry run: true) (only simulation, no real label changes)

What stays local

Python keeps:

- current workflows
- current scripts
- current production behavior
- trigger ownership
- permissions
- harden-runner
- checkout
- config file

No existing Python automation is removed or replaced in this phase.

Acceptance gate

Phase 2 is accepted when:

- the upstream PR is small and reviewable
- the workflow is manual
- dry-run output is visible in Actions logs
- no labels or PR state are changed
- the central parity suite already passes for the relevant behavior
- maintainers can compare intended changes against current review queue behavior

Rollback plan

Close the PR or remove the canary workflow. Existing Python automation remains untouched.

Maintainer review required

Python SDK maintainer approval before merge.

Evidence produced

- upstream PR
- dry-run workflow logs
- comparison notes against existing review-sync behavior

Phase 3: Python Review-Sync Production Pilot

Goal

Enable one low-risk automation for real in one SDK repository after dry-run behavior is accepted.

What changes

The Python canary moves from manual dry-run to controlled write behavior.

- keep `workflow_dispatch` and allow maintainers to trigger write runs manually
- add a limited schedule
- add a narrow event trigger after review

Only review-sync is enabled. No assignment logic is migrated in this phase.

What stays local

Python still keeps workflow ownership and permissions local.

Acceptance gate

Phase 3 is accepted when:

- maintainers approve exact permissions
- action is pinned to a full-length commit SHA
- the central parity suite passes for the shipped behavior
- the caller workflow has an explicit concurrency key
- review-sync writes match expected behavior
- no unrelated labels are changed
- rollback has been tested or documented
- the pilot has a named owner who reviews drift, noise, and unexpected label churn
- the pilot runs for a documented observation window without maintainer complaints
- rollback SLA is documented before enabling writes
- the team agrees on what counts as a failed pilot

Rollback plan

Options:

- disable the workflow
- set dry-run back to true
- revert the canary PR
- pin back to the last known-good full SHA

Maintainer review required

Python SDK maintainers and automation mentor.

Evidence produced

- workflow run logs
- before/after label examples
- issue list of edge cases found
- rollback notes
- parity test results
- observation window summary

Phase 4: Assignment Behavior Mapping

Goal

Understand C++ and Python assignment behavior before extracting shared assignment logic.

What changes

Create a behavior matrix covering:

- supported commands
- skill labels
- priority labels
- assignment limits
- maintainer overrides
- author restrictions
- existing comment text
- label transitions
- stale assignment handling
- race-condition handling
- error messages
- tests and fixtures

What stays local

No assignment workflow changes are made in this phase.

Acceptance gate

Phase 4 is accepted when:

- C++ and Python behavior are documented side by side
- common logic is identified
- repo-specific policy is identified
- test fixtures are listed
- maintainers agree which behavior should be shared and which should remain repo-specific

Rollback plan

No production behavior changes exist, so rollback is not needed.

Maintainer review required

C++ and Python maintainers should review the behavior matrix before implementation.

Evidence produced

- assignment behavior matrix
- proposed config fields
- test fixture list
- migration risk list

Phase 5: Shared Assignment Core

Goal

Implement shared assignment logic only after the behavior matrix is accepted.

What changes

In `sdk-automations`:

- add shared assignment core module
- add config validation for assignment policy
- add tests based on C++ and Python fixtures
- require fixture parity before any upstream assignment write pilot
- add GitHub Action adapter support
- add dry-run support

What stays local

SDK repositories keep:

- command trigger workflow
- `issue_comment` event wiring
- permissions
- harden-runner
- checkout
- config values
- rollout decision

Acceptance gate

Phase 5 is accepted when:

- central tests cover accepted behavior
- fixture parity exists for both Python and C++ assignment expectations where behavior is intended to match
- dry-run output is understandable
- comment text changes are intentional and reviewed
- race-condition protections are preserved
- no SDK repo has switched production assignment yet

Rollback plan

Do not adopt the assignment action until tests and maintainer review pass. If a central release has a bug, patch centrally and issue a new version before SDK adoption.

Maintainer review required

C++ and Python maintainers, because assignment is contributor-facing.

Evidence produced

- central tests
- fixture coverage
- dry-run examples
- release notes

Stretch / Post-Mentorship Expansion Path

The remaining phases are the stretch or continuation path unlocked by the mentorship outputs. By default they remain post-mentorship, because the core program should not overpromise. But the plan should still try to pull a small, reviewable part of this path into the late mentorship window if earlier gates are green, especially for app-readiness work that proves the core is genuinely future-facing.

Post-Mentorship Phase 6: C++ Canary

Goal

Evaluate whether C++ should adopt central actions after the interface is proven through Python.

What changes

Start with simple workflows first. Do not begin with awkward workflows.

Potential candidates:

- simple PR open/update workflows if the central action clearly reduces local duplication
- non-mutating dry-run mode first

Avoid initial migration of:

- `on-pr-review-labels.yaml` if artifact handling is still local and nuanced
- `on-pr-close.yaml` if multi-job behavior would make a wrapper too complex
- assignment workflows until shared assignment core is fully tested

What stays local

C++ keeps:

- event triggers
- `pull_request_target` boundaries
- harden-runner
- checkout refs
- artifact handling
- multi-job structure
- config file

Acceptance gate

Phase 6 is accepted when:

- #1627 investigation has a clear recommendation
- simple and awkward workflow before/after examples are reviewed
- StepSecurity concerns are addressed
- no contributor-facing behavior changes without explicit maintainer approval
- the caller workflow has an explicit concurrency key
- dry-run output is reviewed before writes
- the pilot has a named owner who reviews drift, noise, and false positives
- the observation window and rollback SLA are documented before writes
- the team agrees on what counts as a failed pilot before expansion

Rollback plan

Revert the C++ canary workflow change or pin back to previous local behavior.

Maintainer review required

C++ maintainers. This phase should not proceed from the central repo alone.

Evidence produced

- #1627 recommendation
- before/after workflow examples
- dry-run logs
- maintainer approval notes

Post-Mentorship Phase 7: Wider SDK Rollout

Goal

Expand only after one SDK pilot is stable and maintainers agree that the shared action model is useful.

What changes

Other SDKs may adopt shared automations where they have matching needs.

- `hier-sdk-js`
- `hier-sdk-java`
- `hier-sdk-tck`
- `hier-sdk-rust`
- `hier-sdk-go`
- `hier-sdk-swift`
- `hier-did-sdk-python`

- `hierodid-sdk-js`

What stays local

Each SDK keeps workflow ownership and only opts into automations it actually needs.

Acceptance gate

Phase 7 is accepted when:

- at least one SDK pilot has been stable
- central release process is trusted
- docs cover how a new SDK opts in
- adoption PRs are small and per-repo
- no repo is forced to adopt unrelated policy

Rollback plan

Each SDK can revert its own workflow/config PR or pin back to the last known-good approved SHA.

Maintainer review required

Maintainers for each adopting SDK.

Evidence produced

- adoption checklist
- per-SDK PRs
- run logs
- release notes

Stretch / Post-Mentorship Phase 8: Probot / GitHub App Adapter

Goal

Add centralized webhook orchestration only after the core logic and config contracts are stable, with the explicit aim of making this a late-program stretch target if the earlier phases are ahead of schedule.

What changes

Add a Probot or GitHub App adapter that reuses `packages/core`.

The minimum valuable mentorship outcome for this phase is not full production hosting. It is a thin adapter spike: shared event contract, adapter skeleton, low-risk webhook path, and deployment/runbook outline that prove the core can feed an app without rewriting the logic.

If time and maintainer appetite allow, that spike can be started in the final month of the mentorship. If not, it remains the best-prepared immediate continuation path after November rather than an unexamined future box.

The app may later handle:

- webhook routing
- installation-based authentication
- cross-repo orchestration
- centralized audit logs
- dashboards or reporting
- scheduled jobs that do not naturally belong in a single SDK workflow

What stays local

SDKs should still keep workflow-owned tasks local when GitHub Actions are the better execution environment.

Acceptance gate

Phase 8 is accepted when:

- hosting ownership is clear
- secrets management is clear
- incident response is clear
- webhook signature validation is tested
- rate limit behavior is tested
- central logs are available
- the app reuses existing core logic instead of rewriting it

Rollback plan

Disable app installation or disable specific feature flags. SDK workflow-based action calls can continue independently if needed.

Maintainer review required

Organization maintainers and whoever owns production hosting.

Evidence produced

- app architecture doc
- deployment runbook
- security review
- webhook test fixtures
- audit log examples

Long-Term Workflow Reduction Strategy

- The long-term goal is **not to keep local workflow YAML forever** for every bot automation. The phased action-based model is the safe bridge: it proves the shared core, config schema, parity tests, and rollout discipline while SDK repos still keep their workflow security boundary visible.
- Once the shared core is stable, webhook/API-only automations can move behind a GitHub App or Probot adapter. At that point, many local bot workflows can be retired entirely because the App receives GitHub webhooks directly and uses installation permissions instead of per-repo workflow runs.
- This should apply only to automations that do not need runner execution, checkout, artifacts, or build/test context. CI jobs, artifact workflows, and repo-specific runner tasks should remain in GitHub Actions.

Testing and Validation Strategy Across the Mentorship

Safety and testing should be treated as first-class deliverables, not as cleanup work that happens after implementation. No phase should be considered complete until it passes the testing gate appropriate to that phase.

Layered Testing Strategy

Layer	What it validates	First required roadmap stage	Expected evidence
Config and schema tests	The local policy file is well-formed, versioned, and rejects invalid or unsafe inputs	Milestone 1	Schema validation tests and compatibility notes
Unit tests	Pure business logic such as routing decisions, label selection, policy checks, and state transitions	Milestone 1-2	Coverage reports and deterministic test cases

Integration tests	Action adapters and mocked GitHub interactions across the central review-sync path	Milestone 2	Mocked Octokit/GitHub-context tests and CI pass
Parity fixtures	The shared implementation matches accepted behavior from existing Python workflows and later assignment behavior	Milestone 2, then expanded in Milestone 4-5	Golden fixtures and before/after comparisons
Security tests	Trusted-branch config reads, permission boundaries, full-SHA pinning assumptions, and pull_request_target guardrails	Milestone 1-3	Negative tests and documented security checklist
Failure-path and race tests	Rate limits, retries, timeouts, concurrent comments, and rollback behavior under stress	Milestone 2-4	Chaos-style cases, concurrency cases, and rollback drill results
Canary and pilot observation	Real-world behavior in dry-run and controlled write mode	Milestone 3	Observation logs, named owner review, no-noise summary
Final acceptance tests	Maintainer usability, docs completeness, and rollout readiness	Final handoff	Demo artifacts, docs package, and checklist review

The most important testing rule is that write mode should never rely only on manual confidence. Dry-run logs are useful, but the stronger gate is automated parity plus a controlled pilot with a named owner and rollback plan.

Testing Gates By Roadmap Stage

1. Milestone 1 closes only when config/schema validation, release policy, and governance rules are testable and documented.
2. Milestone 2 closes only when the shared review-sync path reaches the coverage target, parity fixtures pass, and the upstream dry-run PR is small enough to review safely.

3. The midterm is meaningful only if a maintainer can inspect one live dry-run path, its tests, and its rollback logic together.
4. Milestone 3 closes only when the controlled write pilot completes an agreed observation window without unrelated label churn or maintainer pain.
5. Milestone 4 and 5 close only when assignment behavior is fixture-backed and the C++ recommendation is evidence-based, not speculative.
6. The final handoff closes only when the docs, issue backlog, testing artifacts, and maintainer adoption package are all complete enough for someone else to continue the work confidently.

How This Work Opens To The Community

The community should widen the rollout only after each phase has created a stable boundary. That means the first community issues should not be the largest issues. They should be the issues whose output is independently reviewable and does not silently change contributor-facing behavior.

Community Opening Principles

- Open issues only after the architecture or policy boundary for that phase is already decided.
- Start with documentation, schema, fixtures, checklists, and behavior analysis before opening privileged workflow behavior changes.
- Use community issues to widen already-understood work, not to discover architecture live inside production-sensitive repositories.
- Keep contributor-facing assignment behavior and privileged workflow rewiring maintainer-owned until the shared model has already been proven in smaller steps.

Suggested Issue Backlog By Phase

Phase	Safe issues to open	Keep maintainer-owned for now	Why this split exists
Phase 1	Release/pinning docs, schema compatibility notes, caller examples, parity harness setup	Official ownership, branch protection, release control	The boundary is clear enough for supporting work, but governance is still a maintainer decision
Phase 2	Dry-run review checklist, comparison	Upstream workflow behavior changes	The Python canary should still be

	notes, adoption checklist polishing		reviewed as a controlled maintainer change
Phase 3	Very limited; mostly docs or observation support	Real write-mode behavior	A noisy pilot can lose trust quickly, so behavior ownership stays tight
Phase 4	Assignment behavior matrix, comment catalog, label/limit inventory, stale-behavior analysis	Final shared behavior decisions	This is the best point to use contributors for evidence gathering before code is shared
Phase 5	Fixture conversion, validation helpers, test coverage, simple before/after docs	Production C++ contributor-facing changes	Implementation work becomes safer after behavior is agreed and fixtureable

The safest first issue batch is Phase 1 support work plus Phase 4 analysis work. Those are high-leverage areas where contributors can create momentum without quietly changing the behavior that maintainers are responsible for defending.

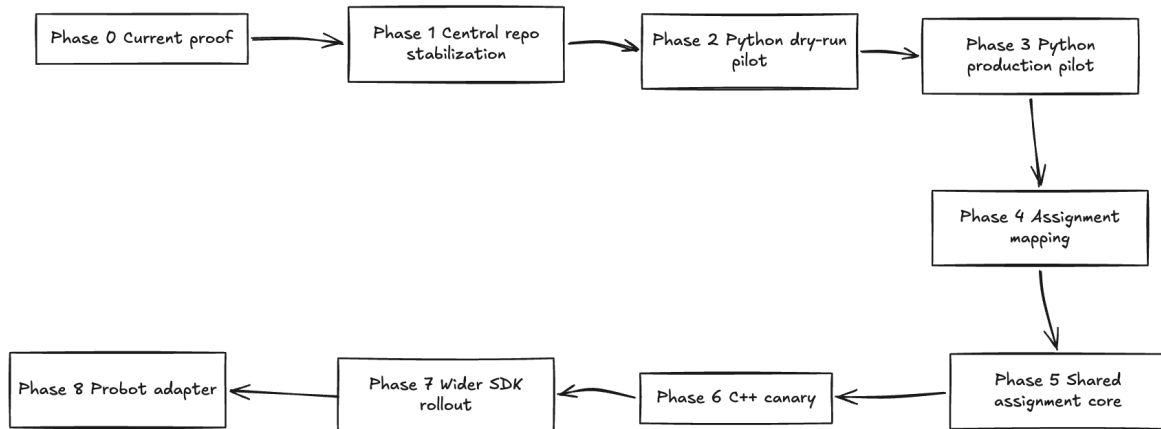
Suggested First Community Issue Batch

1. Document the production full-SHA pinning policy for shared automation callers.
2. Write the schema compatibility and config versioning contract for hiero-automation.*.
3. Build Python review-sync golden fixtures and parity test cases.
4. Write the SDK caller adoption checklist for maintainers.
5. Create the Python/C++ assignment behavior matrix.
6. Catalog assignment comments, transitions, limits, and label interactions.
7. Prepare fixture-harness support tasks for future shared assignment logic.
8. Prepare simple C++ before/after workflow examples for investigation review.

Issues To Delay Until Later

- Full C++ workflow migration issues before the design investigation is accepted.
- Any issue that changes contributor-facing assignment behavior before the Python/C++ behavior matrix is agreed.
- Production Probot hosting or GitHub App issues before the shared action model is stable.
- Wrapper issues whose main value is reducing YAML while hiding harden-runner, permissions, or trust boundaries.

Rollout Timeline View



This timeline is intentionally sequential at the decision points. Some research work can happen in parallel, but production rollout should move through gates.

What Not To Migrate Yet

Do not migrate all C++ workflows now.

Reason: C++ is high-value but sensitive. It uses contributor-facing automation and has workflows with `pull_request_target`, `workflow_run`, artifacts, and multi-job behavior.

Do not hide harden-runner.

Reason: harden-runner visibility is part of the security posture and avoids repeating prior StepSecurity noise.

Do not centralize workflow YAML just for aesthetics.

Reason: workflow YAML contains repository-specific security and event boundaries. Removing duplication is only useful when it removes real maintenance cost.

Do not migrate assignment logic until Python and C++ behavior are compared.

Reason: assignment impacts contributors directly. A central version must preserve accepted behavior and expose repo-specific policy as config.

Do not make Probot the first dependency.

Reason: Probot adds hosting and operations. It should reuse proven core logic later.

Maintainer Decision Points Before Upstream Adoption

The planning value of this document depends on making the remaining dependency decisions explicit. The migration should not pretend those decisions are already settled.

1. Confirm the official long-term home and governance model for the central repository.
2. Confirm that Python review-sync is the first upstream pilot and that assignment logic is intentionally later.

3. Confirm the production pinning rule, config protection rule, and parity-before-write rule as project policy.
4. Confirm which Phase 1 and Phase 4 issue categories can safely be opened to the community immediately.
5. Confirm that C++ remains design/investigation work until Python proves the shared interface and rollout pattern.

Final Recommendation

The clean mentorship reading should therefore be:

- complete Phase 0 before the official program, execute Phases 1-5 during the June 15-November 30 mentorship window, and treat Phases 6-8 as the prepared post-mentorship continuation path **unless maintainers explicitly choose to pull one of them earlier.**

The integrated roadmap removes the cross-reference problem between phases and milestones. Each row now shows the phase, the timebox, the concrete deliverables, and the review gate in one place.

- Within a six-month LFDT mentorship, success should not mean attempting to centralize every workflow. Success should mean proving one shared-action path rigorously, leaving a trustworthy testing and rollout framework behind it, opening the right community issues at the right time, and ending with a maintainer-reviewed recommendation package for the harder C++ and assignment work.
- At the same time, the plan should not read as passive. If the core phases are healthy by the back half of the program, a strong mentorship finish should also leave behind an app-ready shared core and a minimal Probot or GitHub App adapter spike, so the future orchestration layer is demonstrated in outline rather than deferred as a vague promise.
- Proceed with the shared automation migration, but only through phased adoption.
- The next concrete step should be Phase 1: stabilize the central repo for review, add governance and parity gates, and then prepare a small Python upstream dry-run PR. C++ should remain in investigation mode until Python review-sync proves the action interface and the maintainers accept the rollout pattern.
- This path gives the project a working proof without sacrificing security visibility, local repository control, or maintainer confidence.
- The migration should therefore be understood as a staged extraction of reusable behavior, not as a repo-wide teardown: move shared decision logic into the central core, keep local workflow boundaries intact, and delete local implementations only when tests, pilots, and maintainers all agree that the replacement is truly equivalent.